

Card Scanner for iPhone — building and installing

Version 1.3

This folder is a complete, ready-to-open Xcode project. It shares the exact same web UI as the Android app, so both phones behave identically; only the camera, the text recognition and the file writing are native.

Honest note up front: an `.ipa` cannot be produced anywhere except on a Mac running Xcode, because Apple's compiler, the iOS SDK and the code-signing chain are Mac-only. So Android arrives as a finished, installable APK and iOS arrives as source you build once on your Mac. After that first build, installing takes about two minutes.

What you need

Requirement	Notes
A Mac	Any Apple-silicon or Intel Mac, macOS 13 or newer
Xcode 15 or newer	Free from the Mac App Store
An Apple ID	The free one is enough — no paid developer account needed
A USB-C / Lightning cable	To connect the iPhone the first time

With only a free Apple ID the app runs on your own devices and the signature expires after 7 days; you re-run the build to refresh it. A paid Apple Developer account (99 EUR/year) extends that to a year and lets you distribute through TestFlight to colleagues.

Build and install, step by step

1. Copy the whole CardScanner folder to your Mac.
2. Double-click CardScanner.xcodeproj.
3. In the left sidebar click the blue CardScanner project → target CardScanner → tab Signing & Capabilities.
4. Tick **Automatically manage signing**, and under *Team* pick your Apple ID (use *Add an Account...* if the list is empty).
5. Change **Bundle Identifier** from `lt.elmo.cardscanner` to something unique to you, e.g. `lt.integratedoptics.cardscanner`. Apple rejects an identifier already claimed by someone else.
6. Plug in the iPhone, unlock it, tap **Trust** if asked, then choose it in the device menu at the top of the Xcode window.

7. Press ⌘R.

8. The first run stops with *Untrusted Developer* on the phone. On the iPhone go to **Settings > General > VPN & Device Management**, tap your Apple ID, tap **Trust**. Press ⌘R again.

The app now sits on the home screen with the Integrated Optics iO icon.

Optional: a shareable build for colleagues

With a paid developer account:

```
cd CardScanner
xcodebuild -scheme CardScanner -configuration Release \
    -destination 'generic/platform=iOS' \
    -archivePath build/CardScanner.xcarchive archive
xcodebuild -exportArchive -archivePath build/CardScanner.xcarchive \
    -exportOptionsPlist ExportOptions.plist \
    -exportPath build/ipa
```

You supply `ExportOptions.plist` with your team ID and

```
`<key>method</key><string>development</string>` (or `app-store` for TestFlight).
```

Where the exported files go

The app writes `Contact-YYYY-MM-DD.xlsx`, `Address-YYYY-MM-DD.xlsx` and `Cards-YYYY-MM-DD.xlsx` into its own Documents folder, which iOS publishes in the **Files** app under *On My iPhone > Card Scanner*.

To get them into Synology Drive: install **Synology Drive** from the App Store, then either

- tap **Share the exported files** in the app right after exporting and pick Synology Drive, or
- open Files, select the three spreadsheets, and move them into the Synology Drive folder.

The Android build writes to `Documents/CardScanner`, which the Synology Drive Android client can sync automatically. iOS sandboxing does not allow that kind of background folder sync, so on iPhone the hand-off is one deliberate tap. That is an iOS platform limit, not something the app can work around.

What is native and what is shared

Layer	Android	iOS
Screens, parsing, Excel writing	assets/ (HTML + JS)	the very same files in CardScanner/web/
Text recognition	ML Kit, on device	Apple Vision VNRecognizeTextRequest, on device
Camera	ACTION_IMAGE_CAPTURE	UIImagePickerController
Picking an existing photo	ACTION_GET_CONTENT	PHPickerViewController
Storage of the card list	app-private file	WKWebView localStorage
Export destination	Documents/CardScanner	app Documents, visible in Files
Sharing	ACTION_SEND_MULTIPLE	UIActivityViewController
HTTP for the ERPNext sync	URLConnection + a CookieManager	URLSession with a shared HTTPCookieStorage

CardScanner/ios-bridge.js is injected before the page loads and provides the same window.Android object the Android build exposes, which is why not one line of the shared UI had to be forked.

Run ./sync-web-assets.sh any time the Android web files change; it re-copies them into CardScanner/web/.

Vision is asked to read English, Lithuanian, German, French and Polish (iOS 16 and newer). Everything happens on the device — nothing is uploaded unless you switch on AI cleanup in Settings and provide your own API key.

New in 1.3

The web layer is shared with the Android build, so the iPhone app gets the same things: the real Frappe exception instead of a traceback head, Link values settled before a document is sent, read-back verification with a green tick or a red cross next to Organisation, Contact and Address, the card photo attached to the contact, and a supplier created when one is missing. The Integrated Optics mark is the app icon (Assets.xcassets/AppIcon.appiconset/AppIcon-1024.png, 1024 px, opaque as the App Store requires) and the header logo.

MARKETING_VERSION is 1.3 and CURRENT_PROJECT_VERSION is 4. Run ./sync-web-assets.sh and then python3 make_xcodeproj.py after changing anything under the Android assets folder.

New in 1.2 — the ERPNext connection

The whole of the country, region and phone standardisation and the direct ERPNext synchronisation lives in the shared web layer (`geo.js`, `norm.js`, `net.js`, `erp.js`), so the iPhone build behaves exactly like the Android one. See the Android README for what the features do.

Only two things were needed on the native side, and both are already in place:

- `ScannerViewController.httpRequest(method:url:headersJson:body:timeout:reqId:)` — a general request over a single `URLSession` whose cookie storage is shared, so the sid cookie that `/api/method/login` sets is reused by the calls that follow. It allows any method, reports the status code back as `{"status":n,"body":"..."}` through `window.onHttp`, and permits plain http only for a local address. `ios-bridge.js` maps it onto `window.Android`.
- `Info.plist` carries `NSAppTransportSecurity → NSAllowsLocalNetworking` plus a local-network usage string, so an ERPNext instance on the office LAN can be reached at all. iOS shows a one-off "find devices on the local network" prompt the first time; accept it. Public addresses still have to be https.

The older `httpPostJson`, used by the optional AI cleanup, is untouched and still https-only.

Known limitations of this iOS build

- **Not compiled or run here.** This sandbox has no macOS, no Xcode and no iPhone, so the Swift code and the project file were written and statically checked but never executed. That applies to the new `httpRequest` too. The project file was parsed and validated with a `pbxproj` parser, the plists were validated, and the shared web layer was tested in a browser — but expect to fix the odd Xcode warning on first build.
- **Re-reading a stored full-size picture** is supported through the *Scan again* path; the card list itself shows the saved thumbnail rather than re-reading the original file.
- **No automatic folder sync** to Synology, as explained above.